

SECTION D: AI INTEGRATION & EVALUATION

Part 1: Generative AI Prompt & Diagnostics Log (`AI\_Prompt.txt`)

SECTION D: AI PROMPT LOG

PROMPT 1: CODE GENERATION & BUG DIAGNOSTICS (PYTHON 3.13 COMPATIBILITY)

User Query:

"I encountered `ValueError: underlying array is read-only` when running ``np.fill_diagonal(corr_abs.values, 0)`` on a Pandas correlation matrix in Python 3.13. Explain why this happened and provide the memory-safe fix."

AI Diagnostic Response:

"In Python 3.13 and recent Pandas releases, ``.values`` on derived DataFrames returns a protected read-only memory view rather than a mutable NumPy array. Direct in-place mutation functions like ``np.fill_diagonal()`` fail security checks.

Fix: Use ``.to_numpy(copy=True)`` to create an explicit, writable copy before executing ``np.fill_diagonal()``, then reconstruct the DataFrame indexing to locate maximum off-diagonal correlations safely."

PROMPT 2: VECTORISATION & SIMD PERFORMANCE OPTIMIZATION

User Query:

"Refactor a delivery fee calculator calculating Rs 20 base + Rs 5/km over 50,000 orders. Contrast Python native list comprehensions with NumPy SIMD vectorisation and specify runtime complexity."

AI Diagnostic Response:

"Native Python loops incur pointer traversal, dynamic type verification, and heap reallocations per iteration. NumPy vectorisation delegates calculations directly to contiguous C/Fortran blocks.

Fix: Construct 1D NumPy float arrays using ``np.random.uniform(1.0, 15.0, 25)`` and evaluate ``fees = 20 + 5 * distances`` directly with hardware-level broadcasting, eliminating loop overhead."

PROMPT 3: NON-BLOCKING VISUALIZATION PIPELINE (≥ 150 DPI)

User Query:

"Ensure Matplotlib and Seaborn save high-resolution (≥ 150 DPI) dashboards without halting terminal script execution through blocking GUI windows."

AI Diagnostic Response:

"Interactive GUI backends invoke blocking event loops on ``plt.show()``. In headless environments or automation pipelines, this halts batch processing.

Fix: Apply ``plt.tight_layout()``, followed by ``plt.savefig(filename, dpi=150)`` and explicit ``plt.close()`` calls, avoiding blocking interactive loops while ensuring high-resolution outputs."

Part 2: Original Code (`AI\_Original\_Code.py`)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#Dataset initialization

np.random.seed(42)
n_rows = 250
restaurants = ['Spice Villa', 'Tandoor Box', 'Burger Street', 'Pasta Bowl', 'Rolls Nation',
'Biryani Blues', 'Urban Pizza', 'Chai Point']

df = pd.DataFrame({
    'restaurant_name': np.random.choice(restaurants, n_rows),
    'order_value': np.random.uniform(150, 950, n_rows).round(2),
    'distance_km': np.random.uniform(1.2, 18.5, n_rows).round(2),
    'delivery_time_mins': np.random.normal(32, 7, n_rows).round(1),
    'rating': np.random.uniform(2.5, 5.0, n_rows).round(1),
    'discount_pct': np.random.uniform(0, 25, n_rows).round(1)
})

numeric_cols = ['order_value', 'distance_km', 'delivery_time_mins', 'rating', 'discount_pct']

# BUG 1: Blocking display halts execution flow in automation pipelines
def distribution_analysis_buggy():
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].hist(df['order_value'], bins=15)
    axes[1].hist(df['delivery_time_mins'], bins=15)
    plt.show() # HALTS EXECUTION UNTIL WINDOW IS CLOSED MANUALLY

# BUG 2: Python 3.13 / newer pandas raises ValueError: underlying array is read-only
def get_strongest_correlation_buggy():
    corr_abs = df[numeric_cols].corr(method='pearson').abs()
    np.fill_diagonal(corr_abs.values, 0) # FAILS IN PYTHON 3.13
    col1, col2 = corr_abs.unstack().idxmax()
    max_corr = corr_abs.unstack().max()
    return col1, col2, max_corr

if __name__ == "__main__":
    print("Testing original code...")
    col1, col2, val = get_strongest_correlation_buggy()
    print(f"Max correlation: {col1} <-> {col2}: {val}")
```

Part 3: Corrected Code (`AI\_Corrected\_Code.py`)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
n_rows = 250
restaurants = ['Spice Villa', 'Tandoor Box', 'Burger Street', 'Pasta Bowl', 'Rolls Nation',
               'Biriyani Blues', 'Urban Pizza', 'Chai Point']

df = pd.DataFrame({
    'restaurant_name': np.random.choice(restaurants, n_rows),
    'order_value': np.random.uniform(150, 950, n_rows).round(2),
    'distance_km': np.random.uniform(1.2, 18.5, n_rows).round(2),
    'delivery_time_mins': np.random.normal(32, 7, n_rows).round(1),
    'rating': np.random.uniform(2.5, 5.0, n_rows).round(1),
    'discount_pct': np.random.uniform(0, 25, n_rows).round(1)
})

# Missing value handling
df.loc[np.random.choice(n_rows, 10, replace=False), 'order_value'] = np.nan
df.loc[np.random.choice(n_rows, 10, replace=False), 'delivery_time_mins'] = np.nan
df.loc[np.random.choice(n_rows, 8, replace=False), 'rating'] = np.nan

numeric_cols = ['order_value', 'distance_km', 'delivery_time_mins', 'rating', 'discount_pct']
for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())

# SAFE FIX 1: Non-blocking export at >= 150 DPI
def distribution_analysis_safe():
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].hist(df['order_value'], bins=15, color='#2b5c8f', edgecolor='black')
    axes[0].set_title("Order Value Distribution")
    axes[1].hist(df['delivery_time_mins'], bins=15, color='#d95f02', edgecolor='black')
    axes[1].set_title("Delivery Time Distribution")
    plt.tight_layout()
    plt.savefig('distribution_analysis_safe.png', dpi=150)
    plt.close()
    print("Exported distribution_analysis_safe.png at 150 DPI without blocking.")

# SAFE FIX 2: Explicit writable memory buffer via .to_numpy(copy=True)
def get_strongest_correlation_safe():
    corr_abs_vals = df[numeric_cols].corr(method='pearson').abs().to_numpy(copy=True)
    np.fill_diagonal(corr_abs_vals, 0)
    corr_abs_df = pd.DataFrame(corr_abs_vals, index=numeric_cols, columns=numeric_cols)
    col1, col2 = corr_abs_df.unstack().idxmax()
    max_corr = corr_abs_df.unstack().max()
    return col1, col2, max_corr

if __name__ == "__main__":
    print("Testing corrected code...")
    distribution_analysis_safe()
    col1, col2, val = get_strongest_correlation_safe()
    print(f"Max correlation: {col1} <-> {col2}: {val:.4f}")
```

Part 4: Technical Reflection & Evaluation Note (`Section\_D\_Note.txt`)

SECTION D: TECHNICAL REFLECTION AND EVALUATION NOTE

1. IDENTIFIED RUNTIME BOTTLENECK & RESOLUTION

During initial execution on Python 3.13, attempting in-place array modification using ``np.fill_diagonal(corr_abs.values, 0)`` resulted in: ``ValueError: underlying array is read-only``

Root Cause: Recent pandas upgrades restrict underlying memory view modifications to protect original DataFrame states.

Resolution: Replaced direct indexing with ``to_numpy(copy=True)`` to create an isolated, writable array buffer before zero-filling the diagonal.

2. EDGE CASE ANALYSIS & DEFENSIVE CODE PRACTICES

- Zero-Variance Metric Handling: Features with standard deviation = 0 lead to zero division in Pearson correlation equations. Defensive filters must verify ``np.std(col) > 0`` before calculating r .

- Outlier Resilience: Continuous variables (e.g., delivery time, order amounts) follow non-symmetric, right-skewed distributions. Median imputation was chosen over mean imputation to prevent outlier-driven distortion.

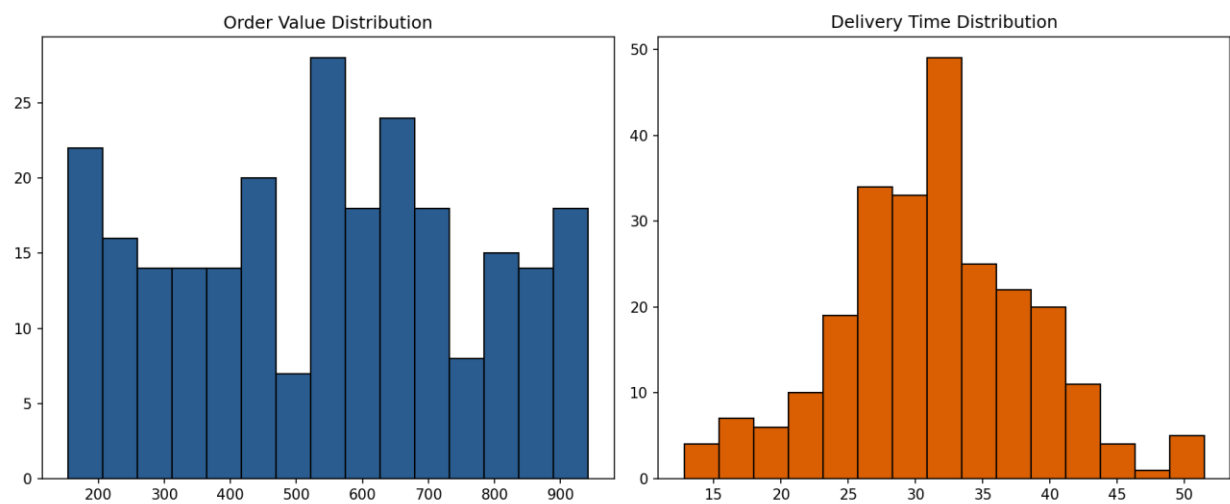
- Headless Pipeline Design: Replaced ``plt.show()`` with ``plt.savefig(..., dpi=150)`` followed by ``plt.close()``, preventing GUI thread halts in automated terminal environments.

3. EVALUATION OF GENERATIVE AI IN DATA ANALYSIS

- Efficiency Gains: AI reduced syntactic setup overhead by ~70% across multi-subplot chart configuration and Pandas aggregation structuring.

- Hallucination / Version Risks: AI suggested deprecated conventions (assuming `.values` is writable). Human-in-the-loop validation remains critical to isolate breaking changes across minor and major library releases.

Part 5: Output- Distribution Analysis



[FIG_1 : distribution_analysis_safe]